

Aim

Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Files

- practical_3
- README.md

Compilation

```
gcc practical_3.c -o practical_3
```

Execution

```
./practical_03
```

Commands to Observe Process States

```
ps -ef | grep practical_03
```

or

```
ps -o pid,ppid,state,cmd
```

Result

The program successfully creates a parent and child process using `fork()`. The PID, PPID, and process states are observed during execution.

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <stdlib.h>
```

```
int main()
{
    pid_t pid;

    printf("Before fork()\n");
    printf("Current Process ID (PID): %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        printf("Fork Failed!\n");
        return 1;
    }
    else if (pid == 0)
    {
        // Child Process
        printf("\n----- Child Process ----- \n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());

        printf("Child is sleeping for 5 seconds...\n");
        sleep(5);

        printf("Child Process Finished.\n");
    }
    else
    {

```

```

// Parent Process

printf("\n----- Parent Process ----- \n");

printf("Parent PID : %d\n", getpid());

printf("Child PID : %d\n", pid);


printf("Parent is sleeping for 20 seconds...\n");

sleep(20);

printf("Parent Process Finished.\n");

}


return 0;

}

```

Output -

Before fork()

Current Process ID (PID): 1216

----- Child Process -----

Child PID : 1217

Parent PID : 1216

Child is sleeping for 5 seconds...

Child Process Finished.

Before fork()

Current Process ID (PID): 1216

----- Parent Process -----

Parent PID : 1216

Child PID : 1217

Parent is sleeping for 10 seconds...

Parent Process Finished.